Hands-on 4: Implement Many-to-One Relationship between Employee and Department

Step 1: Employee Entity

Employee.java

```
package com.cognizant.ormlearn.model;
import java.util.Date;
import javax.persistence.*;
@Entity
@Table(name = "employee")
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "em_id")
    private int id;
    @Column(name = "em_name")
    private String name;
    @Column(name = "em_salary")
    private double salary;
    @Column(name = "em_permanent")
    private boolean permanent;
    @Temporal(TemporalType.DATE)
    @Column(name = "em_date_of_birth")
    private Date dateOfBirth;
    @ManyToOne
    @JoinColumn(name = "em_dp_id")
    private Department department;
    // Getters and Setters
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public double getSalary() {
        return salary;
    }
    public void setSalary(double salary) {
        this.salary = salary;
    }
    public boolean isPermanent() {
        return permanent;
    }
    public void setPermanent(boolean permanent) {
        this.permanent = permanent;
    }
    public Date getDateOfBirth() {
        return dateOfBirth;
    }
    public void setDateOfBirth(Date dateOfBirth) {
        this.dateOfBirth = dateOfBirth;
```

```
    }
    public Department getDepartment() {
        return department;
    }
    public void setDepartment(Department department) {
        this.department = department;
    }
    @Override
    public String toString() {
        return "Employee [id=" + id +
                ", name=" + name +
                ", salary=" + salary +
                ", permanent=" + permanent +
                ", dateOfBirth=" + dateOfBirth + "]";
    }
}
```

Step 2: Department Entity

Department.java

```
package com.cognizant.ormlearn.model;
import javax.persistence.*;
@Entity
@Table(name="department")
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name="dp_id")
    private int id;
    @Column(name="dp_name")
    private String name;
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id=id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name=name;
    }
    @Override
    public String toString() {
        return "Department [id=" + id +
                ", name=" + name + "]";
    }
}
```

Step 3: Skill Entity

```
package com.cognizant.ormlearn.model;
import javax.persistence.*;
@Entity
@Table(name="skill")
public class Skill {
```

```
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name="sk_id")
    private int id;
    @Column(name="sk_name")
    private String name;
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id=id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name=name;
    }
    @Override
    public String toString() {
        return "Skill [id=" + id +
                ", name=" + name + "]";
    }
}
```

Step 4: Repository Interfaces

EmployeeRepository

```
package com.cognizant.ormlearn.repository;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.cognizant.ormlearn.model.Employee;
@Repository
public interface EmployeeRepository
        extends JpaRepository<Employee,Integer>{
}
```

DepartmentRepository

```
package com.cognizant.ormlearn.repository;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.cognizant.ormlearn.model.Department;
@Repository
public interface DepartmentRepository
        extends JpaRepository<Department,Integer>{
}
```

SkillRepository

```
package com.cognizant.ormlearn.repository;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.cognizant.ormlearn.model.Skill;
@Repository
public interface SkillRepository
```

```
        extends JpaRepository<Skill,Integer>{
}
```

Step 5: EmployeeService

```
package com.cognizant.ormlearn.service;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import com.cognizant.ormlearn.model.Employee;
import com.cognizant.ormlearn.repository.EmployeeRepository;
import jakarta.transaction.Transactional;
@Service
public class EmployeeService {
    @Autowired
    private EmployeeRepository employeeRepository;
    @Transactional
    public Employee get(int id){
        return employeeRepository.findById(id).get();
    }
    @Transactional
    public void save(Employee employee){
        employeeRepository.save(employee);
    }
}
```

Step 6: DepartmentService

```
package com.cognizant.ormlearn.service;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import com.cognizant.ormlearn.model.Department;
import com.cognizant.ormlearn.repository.DepartmentRepository;
import jakarta.transaction.Transactional;
@Service
public class DepartmentService {
    @Autowired
    private DepartmentRepository departmentRepository;
    @Transactional
    public Department get(int id){
        return departmentRepository.findById(id).get();
    }
    @Transactional
    public void save(Department department){
        departmentRepository.save(department);
    }
}
```

Step 7: SkillService

```
package com.cognizant.ormlearn.service;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import com.cognizant.ormlearn.model.Skill;
import com.cognizant.ormlearn.repository.SkillRepository;
import jakarta.transaction.Transactional;
@Service
```

```
public class SkillService {
    @Autowired
    private SkillRepository skillRepository;
    @Transactional
    public Skill get(int id){
        return skillRepository.findById(id).get();
    }
    @Transactional
    public void save(Skill skill){
        skillRepository.save(skill);
    }
}
```

Step 8: Autowire Services in OrmLearnApplication

```
private static EmployeeService employeeService;
private static DepartmentService departmentService;
private static SkillService skillService;
```

Inside main()

```
ApplicationContext context =
SpringApplication.run(OrmLearnApplication.class,args);
employeeService=context.getBean(EmployeeService.class);
departmentService=context.getBean(DepartmentService.class);
skillService=context.getBean(SkillService.class);
```

Step 9: Test Get Employee

```
private static void testGetEmployee(){
    LOGGER.info("Start");
    Employee employee = employeeService.get(1);
    LOGGER.debug("Employee : {}",employee);
    LOGGER.debug("Department : {}",employee.getDepartment());
    LOGGER.info("End");
}
```

Call it inside main():

```
testGetEmployee();
```

Output

```
INFO  Start
DEBUG Employee :
Employee[id=1,name=John,salary=50000.0...]
DEBUG Department :
Department[id=1,name=IT]
INFO End
```

Generated SQL

```
select e.*,d.*
from employee e
left outer join department d
```

```
on e.em_dp_id=d.dp_id
where e.em_id=1;
```

Step 10: Add Employee

Add the following method in OrmLearnApplication.java

```
private static void testAddEmployee() {
    LOGGER.info("Start");
    Employee employee = new Employee();
    employee.setName("David");
    employee.setSalary(60000);
    employee.setPermanent(true);
    employee.setDateOfBirth(new Date());
    // Fetch Department with ID = 1
    Department department = departmentService.get(1);
    // Set Department
    employee.setDepartment(department);
    // Save Employee
    employeeService.save(employee);
    LOGGER.debug("Employee Added : {}", employee);
    LOGGER.info("End");
}
```

Call the method inside main()

```
testAddEmployee();
```

Comment other test methods while running this one.

Output

```
INFO  Start
DEBUG Employee Added :
Employee [id=5, name=David, salary=60000.0,
permanent=true, dateOfBirth=2026-06-26]
INFO End
```

Generated SQL

Fetch Department

```
select *
from department
where dp_id=1;
```

Insert Employee

```
insert into employee
(em_name,
 em_salary,
 em_permanent,
 em_date_of_birth,
 em_dp_id)
values
('David',
60000,
1,
'2026-06-26',
```

1);

Verify in Database

Execute:

SELECT * FROM employee;

Expected Result:

em_id em_name em_salary em_dp_id
5 David 60000 1

Step 11: Update Employee

Suppose Employee ID 1 belongs to Department 1.

We'll change it to Department 2.

Add Method in OrmLearnApplication.java

```
private static void testUpdateEmployee() {
    LOGGER.info("Start");
    // Fetch Employee
    Employee employee = employeeService.get(1);
    // Fetch another Department
    Department department = departmentService.get(2);
    // Change Department
    employee.setDepartment(department);
    // Save Updated Employee
    employeeService.save(employee);
    LOGGER.debug("Updated Employee : {}", employee);
    LOGGER.info("End");
}
```

Call inside main()

```
testUpdateEmployee();
```

Output

```
INFO Start
DEBUG Updated Employee :
Employee[id=1,name=John,salary=50000]
INFO End
```

Generated SQL

Fetch Employee

```
select *
from employee
where em_id=1;
```

Fetch Department

```
select *
from department
where dp_id=2;
```

Update Employee

```
update employee
set em_dp_id=2
where em_id=1;
```

Verify Database

Execute:

```
SELECT em_id,
      em_name,
      em_dp_id
FROM employee
WHERE em_id=1;
```

Expected Output

```
em_id  em_name       em_dp_id
1         John    2
```

Complete main() Method (Example)

```
public static void main(String[] args) {
   ApplicationContext context =
         SpringApplication.run(
               OrmLearnApplication.class, args);
   employeeService =
         context.getBean(EmployeeService.class);
   departmentService =
         context.getBean(DepartmentService.class);
   skillService =
         context.getBean(SkillService.class);
   // Uncomment only one test at a time
   // testGetEmployee();
   // testAddEmployee();
   testUpdateEmployee();
}
```

Hands-on 5: Implement One-to-Many Relationship between Employee and Department

Objective is to Implement a One-to-Many relationship where:

* One Department can have many Employees.
* Each Employee belongs to only one Department.

Step 1: Update Department.java

Add the employees field and map it using @OneToMany.

```
package com.cognizant.ormlearn.model;
import java.util.List;
import javax.persistence.*;
@Entity
@Table(name = "department")
public class Department {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "dp_id")
    private int id;
    @Column(name = "dp_name")
    private String name;
    @OneToMany(mappedBy = "department",
            fetch = FetchType.EAGER,
            cascade = CascadeType.ALL)
    private List<Employee> employeeList;
    // Getters and Setters
    public int getId() {
        return id;
    }
    public void setId(int id) {
        this.id = id;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public List<Employee> getEmployeeList() {
        return employeeList;
    }
    public void setEmployeeList(List<Employee> employeeList) {
        this.employeeList = employeeList;
    }
    @Override
    public String toString() {
        return "Department [id=" + id + ", name=" + name + "]";
    }
}
```

Step 2: Verify Employee.java

Ensure the department field already exists (from Hands-on 4).

```
@ManyToOne
@JoinColumn(name = "em_dp_id")
private Department department;
```

No further changes are required in Employee.java.

Step 3: DepartmentService

The service remains the same.

```
package com.cognizant.ormlearn.service;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import com.cognizant.ormlearn.model.Department;
import com.cognizant.ormlearn.repository.DepartmentRepository;
import jakarta.transaction.Transactional;
@Service
public class DepartmentService {
    @Autowired
    private DepartmentRepository departmentRepository;
    @Transactional
    public Department get(int id) {
        return departmentRepository.findById(id).get();
    }
    @Transactional
    public void save(Department department) {
        departmentRepository.save(department);
    }
}
```

Step 4: Add Test Method in OrmLearnApplication.java

```
private static void testGetDepartment() {
    LOGGER.info("Start");
    Department department = departmentService.get(1);
    LOGGER.debug("Department : {}", department);
    LOGGER.debug("Employees :");
    for (Employee employee : department.getEmployeeList()) {
        LOGGER.debug(employee.toString());
    }
    LOGGER.info("End");
}
```

Step 5: Call the Test Method

```
public static void main(String[] args) {
    ApplicationContext context =
            SpringApplication.run(OrmLearnApplication.class, args);
    departmentService = context.getBean(DepartmentService.class);
    testGetDepartment();
}
```

Output

```
INFO  Start
DEBUG Department :
Department [id=1, name=IT]
DEBUG Employees :
Employee [id=1, name=John, salary=50000.0]
Employee [id=2, name=David, salary=60000.0]
Employee [id=3, name=Peter, salary=45000.0]
INFO End
```

Generated SQL

Fetch Department :

```
select *
from department
where dp_id = 1;
```

Fetch Employees :

```
select *
from employee
where em_dp_id = 1;
```

Database Structure

Department Table :

dp_id	dp_name
1	IT
2	HR

Employee Table :

em_id	em_name	em_dp_id
1	John	1
2	David	1
3	Peter	1
4	Alice	2

Hands-on 6: Implement Many-to-Many Relationship between Employee and Skill

Objective is to Implement a Many-to-Many relationship where:

* One Employee can have multiple Skills.
* One Skill can belong to multiple Employees.

Example:

```
Employee
John  ---- Java
      \--- Spring
      \--- SQL
David ---- Java
      \--- Python
```

Step 1: Update Employee.java

Add a List<Skill> field.

```
package com.cognizant.ormlearn.model;
import java.util.List;
import javax.persistence.*;
```

```
@Entity
@Table(name = "employee")
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "em_id")
    private int id;
    @Column(name = "em_name")
    private String name;
    @Column(name = "em_salary")
    private double salary;
    @Column(name = "em_permanent")
    private boolean permanent;
    @Temporal(TemporalType.DATE)
    @Column(name = "em_date_of_birth")
    private java.util.Date dateOfBirth;
    @ManyToOne
    @JoinColumn(name = "em_dp_id")
    private Department department;
    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(
        name = "employee_skill",
        joinColumns = @JoinColumn(name = "es_em_id"),
        inverseJoinColumns = @JoinColumn(name = "es_sk_id")
    )
    private List<Skill> skillList;
    // Getters and Setters
    public List<Skill> getSkillList() {
        return skillList;
    }
    public void setSkillList(List<Skill> skillList) {
        this.skillList = skillList;
    }
    // Other getters and setters
}
```

Step 2: Update Skill.java

```
package com.cognizant.ormlearn.model;
import java.util.List;
import javax.persistence.*;
@Entity
@Table(name = "skill")
public class Skill {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "sk_id")
    private int id;
    @Column(name = "sk_name")
    private String name;
    @ManyToMany(mappedBy = "skillList",
            fetch = FetchType.EAGER)
    private List<Employee> employeeList;
    // Getters and Setters
    public List<Employee> getEmployeeList() {
        return employeeList;
    }
    public void setEmployeeList(List<Employee> employeeList) {
        this.employeeList = employeeList;
```

```
    }
    // Other getters and setters
}
```

Step 3: Database Tables

Employee

```
employee
--------
em_id
em_name
em_salary
em_dp_id
```

Skill

```
skill
-----
sk_id
sk_name
```

Employee_Skill (Join Table)

```
employee_skill
--------------
es_em_id
es_sk_id
```

Example Data

es_em_id	es_sk_id
1	1
1	2
2	1
2	3

Step 4: EmployeeService

```
@Service
public class EmployeeService {
    @Autowired
    private EmployeeRepository employeeRepository;
    @Transactional
    public Employee get(int id) {
        return employeeRepository.findById(id).get();
    }
    @Transactional
    public void save(Employee employee) {
        employeeRepository.save(employee);
    }
}
```

Step 5: SkillService

```
@Service
public class SkillService {
    @Autowired
    private SkillRepository skillRepository;
    @Transactional
    public Skill get(int id) {
        return skillRepository.findById(id).get();
    }
    @Transactional
    public void save(Skill skill) {
        skillRepository.save(skill);
    }
}
```

Step 6: Test Employee with Skills

Add the following method to OrmLearnApplication.java.

```
private static void testGetEmployee() {
    LOGGER.info("Start");
    Employee employee = employeeService.get(1);
    LOGGER.debug("Employee : {}", employee);
    LOGGER.debug("Skills:");
    for (Skill skill : employee.getSkillList()) {
        LOGGER.debug(skill.toString());
    }
    LOGGER.info("End");
}
```

Step 7: Test Skill with Employees

```
private static void testGetSkill() {
    LOGGER.info("Start");
    Skill skill = skillService.get(1);
    LOGGER.debug("Skill : {}", skill);
    LOGGER.debug("Employees:");
    for (Employee employee : skill.getEmployeeList()) {
        LOGGER.debug(employee.toString());
    }
    LOGGER.info("End");
}
```

Step 8: Main Method

```
public static void main(String[] args) {
    ApplicationContext context =
            SpringApplication.run(OrmLearnApplication.class,args);
    employeeService = context.getBean(EmployeeService.class);
    skillService = context.getBean(SkillService.class);
    testGetEmployee();
    // testGetSkill();
}
```

Output

Employee

```
INFO Start
Employee :
Employee[id=1,name=John]
Skills:
Skill[id=1,name=Java]
Skill[id=2,name=Spring]
Skill[id=3,name=SQL]
INFO End
```

Skill

```
INFO Start
Skill :
Skill[id=1,name=Java]
Employees
Employee[id=1,name=John]
Employee[id=2,name=David]
INFO End
```

Generated SQL

Fetch Employee

```
select *
from employee
where em_id = 1;
```

Fetch Skills

```
select s.*
from skill s
join employee_skill es
on s.sk_id = es.es_sk_id
where es.es_em_id = 1;
```

Fetch Skill

```
select *
from skill
where sk_id = 1;
```

Fetch Employees

```
select e.*
from employee e
join employee_skill es
on e.em_id = es.es_em_id
where es.es_sk_id = 1;
```